***Abstract*—Text-to-SQL agents fail where aggregate accuracy cannot see: a synonym never learned, a definition never read, a join template never questioned. We describe—and execute—a closed-loop evaluation framework for such agents (here, deterministic rule-based agents): six deterministic stages (S1–S6), per-failure stage attribution, a mapping from stages to four enterprise artifact layers (metadata, glossary, query templates, result verification) as repair surfaces, and verified repair by re-execution counting repaired (R^+), regressed (R^-), and net (R^Δ) questions. On a 12-question synthetic retail suite, an 88-question Spider slice, and a 100-question BIRD Mini-Dev slice, localized repair raises execution accuracy from 5/12 (Wilson 95% CI [0.193, 0.681]) to 12/12 ([0.758, 1.000]; R^+=7, R^-=0), 10/88 to 15/88 (R^+=5, R^-=0), and 0/100 to 6/100 (R^+=6, R^-=0). A blind-repair comparison on Spider contrasts stage-guided repair (R^Δ=+5) with an aggressive undiagnosed 21-pair synonym policy (R^Δ=−5; EX 10/88→5/88, 6 regressed)—different policies, not the same edits two ways. Matched-budget blind baselines match localized repair on both slices (A9: R^+=5, R^-=0; A10: R^+=6, R^-=0), showing diagnosis identified the repair mechanism, not the specific pairs. Classifying 25 published LLM-agent errors into S1–S6 (A11) shows the taxonomy partially transfers as a descriptive vocabulary (24/25 classifiable, 96%). Dev-derived fixes generalize to held-out questions (Spider R^+=4, R^-=0; BIRD R^+=4, R^-=0; Wilson 95% [0.036, 0.212] and [0.032, 0.188]). At full-dev scale (n=983 and n=500), accuracy moves 79/983→95/983 (R^+=27, R^-=11) and 2/500→8/500 (R^+=6, R^-=0)—including 11 regressions the slices hid, caught by the R^+/R^-/R^Δ accounting. We claim the executed integration, not the novelty of its components.**

***Index Terms*—text-to-SQL, agent evaluation, failure localization, execution accuracy, verified repair**

Closing the Loop: Stage-Wise Evaluation, Failure Localization, and Verified Repair for Text-to-SQL Agents

Prabhu Rangaraju and Ramesh Kumar Nanjundappan
Independent Researcher
prab.rangaraju@gmail.com, ramesh.nanjundappan@gmail.com

I. Introduction

Text-to-SQL agents are deployed into enterprises where the artifacts that govern their behavior are not weights but documents: table metadata, business glossaries, approved query templates, and result verification rules. As AI agents become participants in organizational workflows, these artifact layers are organizational knowledge the agent must get right—a wrong glossary definition or stale synonym is a wrong answer delivered with confidence. When such an agent returns a wrong number, the production question is "which artifact is wrong?"—and fixing the wrong artifact is how well-meaning repairs silently regress other questions. Aggregate scores (Spider [1], BIRD [2]) and query-by-query patching both miss this: neither tells an operator which artifact to change, or whether the change repaired only what it targeted.

This paper describes a closed-loop evaluation framework for that gap and executes one end-to-end pass. *Stage-wise evaluation* decomposes generation into six deterministic stages (S1–S6); *failure localization* attributes each failure to one priority-ordered stage label; *artifact-layered repair* maps stages to the four artifact layers so a label proposes a concrete edit; *verified repair* re-executes the repaired agent and counts repaired (R^+), regressed (R^-), and net (R^Δ) questions, admitting a repair only when re-execution confirms it.

The claimed contribution is deliberately narrow: the executed integration of enterprise artifacts and repair layers, deterministic stage attribution, a stage-to-artifact repair mapping, and separate-verifier before/after verification on limited slices with rule-based agents. We claim no novelty for the components individually—each has precedents (§VII). Stage labels are priority-ordered attributions, not established causes. All agents are deterministic and rule-based; the repair evidence does not transfer to LLM-based agents, though the S1–S6 taxonomy partially transfers as a descriptive vocabulary for published LLM-agent errors (A11). Longitudinal learning and expert workflow capture are unimplemented future work.

We instantiate on three slices (synthetic 5/12→12/12; Spider 10/88→15/88; BIRD 0/100→6/100), warn via a blind-repair comparison (R^Δ=+5 vs. -5), and probe the claims with eleven strengthening analyses (A1–A11)—reporting the uncomfortable findings rather than hiding them: overlapping confidence intervals, superadditive layer interaction (isolated sum 5 vs. cumulative 7), and order-dependent labels (34/78, 43.6%, flip S2→S3).

II. The Framework

A. Six deterministic stages

We model text-to-SQL generation as a pipeline of six deterministic stages: the same inputs always produce the same outputs, so attribution is reproducible. **S1** intent understanding maps the question to a normalized intent (entities, measure, filters). **S2** schema linking maps intent entities to tables and columns, failing as SCHEMA-LINK-MISS. **S3** semantic grounding applies the business glossary (definitions, domain rules, evidence predicates), failing on definition violations and join errors. **S4** query construction assembles SQL from templates (aggregation/filter shapes, join patterns), failing as AGG-FILTER-ERROR. **S5** execution runs the SQL (EXECUTION-ERROR). **S6** result verification compares against the gold result (EMPTY-RESULT, WRONG-RESULT).

B. Failure modes and priority-ordered attribution

Each failed question gets one (stage, mode) pair: schema-link misses → S2, semantic violations and join errors → S3, aggregation or filter errors → S4, execution errors → S5, and empty or wrong results → S6 (formal mode names are listed in §II-D). The diagnoser checks modes in a fixed priority order (S5, EX guard, S2 gaps, S3 checks, S3 join coverage, S4 shape, S6 result)—assigning the first match; sub-orders are slice-dependent. These labels are attributions under a stated priority order—not experimentally established causes. An S6 label says verification failed before any earlier check fired; it does not say verification "caused" the failure.

C. Enterprise artifacts and repair layers

Repairs edit four artifact layers, each owned by a different enterprise role. **L1** metadata: table/column synonyms and descriptions (data stewards; repairs S2 misses, e.g. "turnover"/"sales"→`revenue`). **L2** glossary: business term definitions and semantic rules (domain experts; repairs S3 violations, e.g. "repeat buyer" = >1 order). **L3** query templates: approved aggregation, filtering, and join shapes (analytics engineers; repairs S3 join and S4 construction errors). **L4**

result verification: execution guards and plausibility rules (QA; repairs S6 issues).

D. Stage-to-layer mapping

The mapping from stage labels to repair layers is fixed and public:

Stage	Mode	Repair layer
S2	SCHEMA-LINK-MISS	L1 Metadata
S3	SEMANTIC-DEFINITION-VIOLATION	L2 Glossary
S3	JOIN-ERROR	L3 Templates
S4	AGG-FILTER-ERROR	L3 Templates
S5	EXECUTION-ERROR	L3 Templates
S6	EMPTY-RESULT, WRONG-RESULT	L4 Verification

Given a failure's (stage, mode) pair, the mapping proposes exactly which artifact to edit. The repair loop is: run, diagnose, map, edit, re-execute, count.

III. VERIFIED REPAIR AND ACCOUNTING

A repair is a before/after artifact pair, re-executed on the same slice with the same harness. A question counts as *repaired* (R^+) if EX goes $0 \to 1$ and as *regressed* (R^-) if it goes $1 \to 0$; the *net* is $R^\Delta = R^+ - R^-$. A repair is admitted only when re-execution confirms it and is reported with both counts, never the net alone. In the three main runs R^-=0 throughout, so the gate functioned as a reporting discipline rather than a selection filter; the A1 blind run is where the both-counts rule matters, recording 6 regressions.

Two guards keep the accounting honest: the *EX guard* (only EX=0 questions diagnosed) and *empty-gold exclusions* (gold SQL returning no rows excluded). Both guards are ablated in §VI.

IV. INSTANTIATION: AGENTS AND BENCHMARK SLICES

A. Rule-based agents

All agents are deterministic and rule-based: fixed lexical parsing, stemmed schema matching, glossary predicates, template-assembled SQL over SQLite. Agent v1.0 is the baseline; v1.1 is v1.0 plus the stage-proposed artifact edits. No language model is used.

B. Synthetic retail suite (n=12)

A 12-question suite over a synthetic retail database (customers, orders, order_items, products), each question exercising a known artifact gap; database, glossary, questions, and gold SQL were authored for this study. The v1.0 glossary deliberately lacks "repeat buyer" (Q12), so its S3 attribution is by construction. Q04 has column-link F1 of 0.00 yet EX=1: schema-link scores and execution success can diverge.

C. Spider slice (n=88)

First 45 of `concert_singer` and `car_1`, minus CS-14/CS-15 (empty-gold exclusion): $n = 88$. Random 10-question gold-derivation sample hand-checked: 10/10 agreement. Repairs: plural-table stemming, four synonyms, subsumption pruning, columnless-table guard.

D. BIRD Mini-Dev slice (n=100)

A deterministic seed-42 sample of the BIRD Mini-Dev package[1]: all 30 `california_schools` questions, all 32 `financial` questions, and the first 38 of a seed-42 shuffle of the question-ID-sorted `thrombosis_prediction` list, for $n = 100$. Every sampled gold SQL executed to non-empty rows; there were no exclusions. We use a local EX harness, not the official evaluator, so BIRD numbers are not leaderboard-comparable. The repair was a query template built around evidence predicates, targeting WRONG-RESULT/EMPTY-RESULT (S6) failures (per-layer isolation: template 6/0/+6; metadata and glossary 0/0/0).

V. RESULTS

Synthetic (5/12→12/12). Baseline 5/12 (Wilson 95% CI [0.193, 0.681]; failures S2=1, S3=4, S4=1, S5=1). Metadata synonyms (L1), glossary definitions (L2), and a join/aggregation template (L3) gave R^+=7 (Q03, Q05, Q06, Q08, Q09, Q11, Q12), R^-=0: post-repair EX 12/12 (Wilson 95% CI [0.758, 1.000]). **Spider (10/88→15/88).** Baseline 10/88; failures concentrated in schema linking (S2=70, S3=1, S4=1, S6=6), expected on real schemas with lexical variation. Repair gave R^+=5, R^-=0: post-repair EX 15/88. **BIRD (0/100→6/100).** Baseline 0/100; failures spread across stages (S2: 48, S3: 17, S4: 15, S6: 20) and modes (48 schema-link misses, 17 semantic-definition violations, 15 agg-filter errors, 4 empty results, 16 wrong results). An evidence-predicate template (L3) repaired 6 questions (5 S6/WRONG-RESULT, 1 S2/SCHEMA-LINK-MISS): R^+=6, R^-=0, post-repair EX 6/100. A substring defect found while validating (`male` inside `female`, misfiring glossary detection) was fixed with word-boundary detection as a baseline correction on both runs—0 contribution to the delta (without it only 5 repair: contradictory `SEX='F' AND SEX='M'` predicates block Q1270).

A. Held-out repair evaluation

Diagnosing and repairing on the same slice means R^-=0 partly reflects the repair author having seen the slice. To test generalization, we split each real-benchmark slice into dev and held-out *before* diagnosis (seed 42; Spider 44/44 and BIRD 50/50, stratified by database), re-derived fixes from the dev failures only, and evaluated on the held-out questions. Derivation scripts were structurally blinded (dev results only), though the analyst had seen the full-slice results—a procedural control, not analyst amnesia. The n=12 suite was not split: 6/6 cannot distinguish generalization from noise.

Slice	Split	Dev $R^+/R^-/R^\Delta$	Held-out $R^+/R^-/R^\Delta$	Held-out rate	Wilson 95% CI
Spider	44/44	1/0/1	4/0/4	4/44	[0.036, 0.212]
BIRD	50/50	2/0/2	4/0/4	4/50	[0.032, 0.188]

On the Spider split, dev failures (S2: 36, S4: 1, S6: 2) re-derived the metadata fix: 3 of 4 synonyms and both

[1]Data-license note: the BIRD dataset is distributed under CC BY-NC 4.0, per the dataset authors' clarification (DAMO-ConvAI issue #30; the Mini-Dev README badges state CC BY-SA 4.0). This academic, non-commercial evaluation of an attributed 100-question slice complies with the license.

guards (R^{Δ}=−4 when dropped: load-bearing). On BIRD, dev re-derived 2 of 9 synonyms, two glossary definitions, and the evidence-predicate template; dev single-fix ablations reproduce the marginal pattern (template-only 2/0/2; metadata/glossary 0/0/0). Dev-derived fixes recover all 5/5 Spider and 6/6 BIRD questions the full-slice fixes had repaired. Held-out R^{-}=0 on both slices, but the sets are small (wide CIs) and held-out R^{+} exceeding dev R^{+} is luck of the split. The fixes remain hand-authored; on BIRD the repairing element is a general mechanism with no data-derived parameters, so the sharper generalization test is the Spider arm.

VI. Strengthening Analyses (A1–A11)

Eleven analyses probe the results, as runnable scripts in `strengthening/` (`exp_a*.py`; 76-test suite in `tests.py`); all numbers regenerate from raw outputs.

a) A1: Localized vs. blind repair.: On Spider we compare two policies. *Localized*: stage-guided v1.1 metadata repair (plural stemming for table matching only, four curated synonyms, subsumption pruning, columnless/table guard). *Blind*: aggressive undiagnosed—stemming on tables and columns, a programmatic 21-pair synonym map with no diagnosis, no pruning or guards. Localized: R^{+}=5, R^{-}=0, R^{Δ}=+5; blind: R^{+}=1, R^{-}=6, R^{Δ}=−5 (EX 10/88 → 5/88), regressing CS-10, CS-11, CS-20, CS-21, CA-38, CA-39. Two different policies, not the same edits two ways: undiagnosed repair can be net-negative—and the accounting catches it—but the comparison does not isolate localization as the causal factor, since the policies differ in aggressiveness as well as guidance. Figure 3 shows the comparison; a matched-budget baseline follows in A9.

b) A2: Test-suite accuracy.: EX under row-duplication and 70% subsampling mutants matched EX-single exactly (synthetic 5/12→12/12; Spider 10/88→15/88); zero "accidentally right" questions. The mutants are mild; no BIRD A2 run, no A2 claim there.

c) A3: Efficiency.: Agent vs. gold SQL runtime (median of five runs): originally 13/15 timed questions within 2× of gold (CS-00 2.18×, CA-18 3.68×)—sub-millisecond SQLite times dominated by machine noise; a rerun found all 15 within 2×. Reported qualitatively: agent SQL runs in the same order as gold. No latency claim beyond this.

d) A4: Guard and ordering.: Removing the EX guard (diagnoser run on 10 EX=1 questions) produced 10/10 spurious diagnoses: load-bearing. Swapping adjacent check order changed 34/78 Spider labels (43.6%), all S2→S3: labels are order-dependent attributions, not causes.

e) A5: Second agent variant.: Agent-B, deliberately degraded: EX 5/88 with table-link F1 0.7165 vs. Agent-A's 0.4572 (S2=57, S3=2, S4=1, S5=7, S6=16)—better schema linking yet worse EX, failures concentrated later: the dissociation stage distributions reveal. Literal-matching extensions fired on 2/88 (ΔF1 = $+0.004$).

f) A6: Uncertainty.: Percentile bootstrap (10,000 resamples, seed 42) 95% intervals: Spider 10/88 → [0.057, 0.182], 15/88 → [0.102, 0.250] (overlap: the +5 gain is not statistically separated at $n = 88$); BIRD 0/100 → [0.000, 0.000], 6/100 → [0.020, 0.110] (non-overlapping, but against a degenerate 0/100 baseline—not comparable with other gains). Gains are reported as observed, not significant.

g) A7: Layer interaction.: On synthetic, isolated layers gave metadata 0/0/0, glossary 4/0/+4, template 1/0/+1 (R^{+}/R^{-}/R^{Δ}): the isolated sum (5) falls short of the cumulative (7)—2 failures needed multiple layers. On BIRD, only the evidence predicate template fired (6/0/+6); metadata and glossary in isolation fixed nothing. The stage-to-layer mapping only partially predicts which layer repairs which failure.

h) A8: Full-dev scale-up.: Frozen v1.0/v1.1 on full Spider dev (1,034 questions, 20 databases; 51 excluded—gold SQL fails or returns no rows; n=983) and 500-question BIRD mini-dev (11 databases, no exclusions). Spider dev: 79/983 (8.0%, Wilson 95% [0.065, 0.099]) → 95/983 (9.7%, [0.080, 0.117]); R^{+}=27, R^{-}=11 (R^{Δ}=+16); 22 of 27 repairs on databases unseen during derivation (flight_2: 11). All 11 regressions occurred off-slice from generic components: a slice-clean repair (R^{-}=0) can regress at scale—the trade-off the accounting quantifies. BIRD mini-dev: 2/500 (0.4%, [0.001, 0.015]) → 8/500 (1.6%, [0.008, 0.031]); R^{+}=6, R^{-}=0 (R^{Δ}=+6), all 6 on slice databases. Intervals overlap on both: a confirmed mechanism with quantified trade-offs, not statistically separated gains.

i) A9: Matched-budget blind baseline.: The A1 blind differed in budget and aggressiveness as well as guidance (21 vs 4 pairs, column stemming, no guards). A matched-budget blind—same 4-synonym budget, table-only stemming, same guards, pairs chosen by a diagnosis-free frequency rule—achieved R^{+}=5, R^{-}=0 (R^{Δ}=+5), EX 15/88 (Wilson 95% CI [0.106, 0.262]): the identical five questions as localized. Stemming plus guards with zero synonyms also reaches 15/88: the pairs change nothing; plural stemming carries the repair. Diagnosis identified the mechanism (table-name plural handling), not the pairs; the old blind's regressions came from aggressiveness, not lack of diagnosis.

j) A10: Matched-budget blind baseline on BIRD.: A matched-budget blind—same 9-synonym budget, same 3 glossary definitions (forced identical: catalog holds 7, v1.0 ships 4), same S4 enforcement and evidence parser, differing only in synonym selection (diagnosis-observed vs. frequency-ranked, no failure information)—fixed the same 6 questions, R^{+}=6, R^{-}=0 (EX 6/100, Wilson 95% CI [0.028, 0.125]), identical to localized. The pairs do not move EX; the content-agnostic evidence parser carries the gains. Diagnosis mattered architecturally—identifying unparsed evidence as the gap—not at the pair level.

k) A11: Transfer to LLM-agent errors.: Classifying 25 published LLM text-to-SQL errors (Shen et al.; NL2SQL-BUGs) into S1–S6: 24/25 (96%) classify (S1: 1, S2: 3, S3: 4, S4: 7, S5: 5, S6: 4); the unclassifiable case (prompt-identical non-determinism) is process-level. Caveats: LLM errors spread S4/S5/S6 (S2 only 12%) vs. our S2-dominated agents; the priority order masks root causes (a hallucinated column routes to S5); 39.9% of Shen et al.'s errors resisted

their own taxonomy. The taxonomy transfers as a descriptive vocabulary, but LLM attribution needs revised priority rules—and S5→L3 is the wrong repair surface for hallucinations.

VII. Related Work

a) Benchmarks and enterprise text-to-SQL.: Spider [1] and BIRD [2] are the standard EX benchmarks; Spider 2.0 [5] extends evaluation to 632 enterprise workflows; SQLBench [6] evaluates pipeline subtasks including schema linking. We consume these benchmarks as slices and contribute no benchmark.

b) Error taxonomies and stage-wise diagnosis.: NL2SQL-BUGs [7] benchmarks semantic-error detection with a two-level taxonomy; Shen et al. [8] categorize in-context-learning errors and propose MapleDoctor; ErrorLLM [9] models errors with dedicated tokens to guide refinement; ToolRobustBench [10] diagnoses tool-agent failures stage-wise; DAC [11] decomposes correction into entity linking and skeleton parsing. Our six stages follow this lineage; the taxonomy is not our novelty.

c) Localized correction and verified repair.: SafeQL [12], SQLGovernor [13], and EGRefine [14] correct the erroneous component rather than regenerating queries. Tool-SQL [3] ties tools to SQL clauses for targeted refinement—the closest precedent for our targeted-vs-blind contrast. QueryAgent [15] does stepwise self-correction with error-type-specific guidelines; SQLFixAgent [16] runs a detect–repair loop (reviewer, crafter, refiner) with gains on five benchmarks; SQLDriller [17] checks NL-to-SQL mappings against counterexample instances; Alrashed et al. [18] formalize verification for LLM text-to-SQL (roundtrip critique, synthetic execution); execution-guarded re-execution also appears in the self-healing SQL Query Engine [19]. SQL-of-Thought [4] routes failures through a taxonomy-guided correction loop—the nearest architectural precedent for stage decomposition with guided correction. Fix/break accounting is standard (e.g., SQLENS [20] reports $N_{fix} = 66$, $N_{break} = 16$); our $R^+/R^-/R^\Delta$ follows the convention. Reflect-SQL [21] and Mohr et al. [22] decompose generation into typed stages with stage-localized violations, closest to our setting; the cost-aware agentic architecture [23] and PV-SQL [24] pair probing with verification over enterprise schemas.

Two public, non-peer-reviewed implementations overlap this space. ReskillSQL[2] executes diagnose→route→repair→validate with regression rollback (self-reported BIRD Mini-Dev 58/100→67/100, 3 regressions); My DB Mate[3] builds the enterprise context layer (glossary, schema annotations, governed metrics) with eval gates. Self-reported numbers; included because they execute the loop outside the archival literature.

[2]ReskillSQL, https://github.com/jxwfdpt42t-ship-it/reskillsql (accessed 2026-09-29); not peer-reviewed.

[3]My DB Mate, https://github.com/phuc-nt/my-db-mate (accessed 2026-09-29); not peer-reviewed.

d) What we add.: The archival literature does not report, to our knowledge, the executed combination of (i) a fixed public stage-to-artifact-layer mapping (S2→L1, S3→L2, S4/S5→L3, S6→L4); (ii) fully deterministic priority-ordered attribution with no LLM in the loop; and (iii) same-slice $R^+/R^-/R^\Delta$ accounting backed by held-out evaluation of dev-derived repairs. That combination—artifact-layered, deterministic, verified—is the contribution; the components are not new.

VIII. Limitations

The agents are deterministic and rule-based; we make no transfer claim to LLM-based agents. Slice samples are small (12, 88, 100); the A8 scale-up (n=983 and n=500) quantifies trade-offs, but gains are not statistically separated at any scale (all intervals overlap) and absolute accuracy remains low. Stage labels are order-dependent attributions, not causes (A4). The stage-to-layer mapping only partially predicts repair locations (A7). The BIRD 0/100 baseline is degenerate, so the +6 gain should not be compared with the Spider or synthetic gains. BIRD numbers use a local EX harness, not the official evaluator, and are not leaderboard-comparable. Longitudinal learning and expert-workflow capture remain unimplemented future work. These limits bound all quantitative statements.

IX. Conclusion

We described and executed a closed-loop evaluation framework for text-to-SQL agents: per-failure stage attribution, stage-to-artifact-layer mapping, artifact repair, and verified repair with repaired/regressed accounting. On three slices with rule-based agents, stage-guided repair improved execution accuracy, and dev-derived repairs generalize to held-out questions (Spider 4/0/4, BIRD 4/0/4; §V-A). Full-dev scale-up (A8) moved 79/983→95/983 (R^+=27, R^-=11) and 2/500→8/500 (R^+=6, R^-=0), surfacing 11 regressions the slices hid—caught by the accounting, not hidden by aggregate accuracy. An aggressive undiagnosed policy regressed six Spider questions, showing undiagnosed lexical repair can hurt; a matched-budget blind (A9) matched localized repair exactly (R^+=5, R^-=0), bounding what localization contributes.

Reproducibility

All experiments are deterministic. The repository contains the framework source, benchmark slices, raw outputs, and the figure-generation script; strengthening scripts A1–A10 and the A11 classification data are included, and the test suite (`strengthening/tests.py`) passes 76/76.

AI Disclosure

The authors used the AI tool Muse (Muse Spark 1.3, Meta) for drafting, grammatical editing, and code validation. The contents are reviewed, edited, and attested for accuracy by the authors.

Acknowledgments

The authors thank the open-source providers of the Spider and BIRD benchmarks.

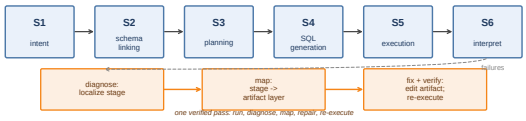


Fig. 1. The closed loop: run, diagnose (stage attribution), map (stage → artifact layer), repair, and verify (re-execute; count repaired vs. regressed).

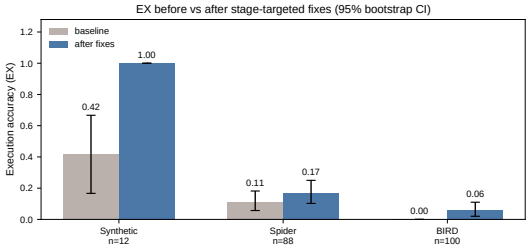


Fig. 2. Execution accuracy before and after localized repair, with 95% bootstrap intervals (10,000 resamples). Intervals overlap on Spider; gains are reported as observed.

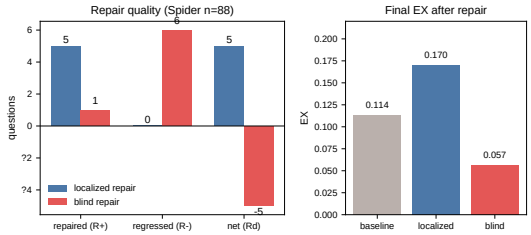


Fig. 3. Localized vs. blind repair on Spider. Stage-guided: R^{Δ}=+5; aggressive undiagnosed policy: R^{Δ}=−5 (6 regressed); matched-budget blind (A9): R^{Δ}=+5.

References

[1] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *EMNLP*, 2018.

[2] Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems*, volume 36, pages 42330–42357, 2023.

[3] Zhongyuan Wang, Richong Zhang, Zhijie Nie, and Jaein Kim. Tool-assisted agent on SQL inspection and refinement in real-world scenarios. arXiv:2408.16991, 2024.

[4] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. SQL-of-Thought: Multi-agentic text-to-SQL with guided error correction. arXiv:2509.00581, 2025. Accepted at the NeurIPS 2025 DL4C workshop.

[5] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. In *ICLR*, 2025. arXiv:2411.07763.

[6] Bin Zhang, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Chi Harold Liu, Zhiwei Xu, Guoliang Fan, Rui Zhao, Ziyue Li, and Hangyu Mao. SQLBench: A comprehensive evaluation for text-to-SQL capabilities of large language models. arXiv:2403.02951, 2024.

[7] Xinyu Liu, Shuyu Shen, Boyan Li, Nan Tang, and Yuyu Luo. NL2SQL-BUGs: A benchmark for detecting semantic errors in NL2SQL translation. arXiv:2503.11984, 2025.

[8] Jiawei Shen, Chengcheng Wan, Ruoyi Qiao, Jiazhen Zou, Hang Xu, Yuchen Shao, Yueling Zhang, Weikai Miao, and Geguang Pu. Understanding, detecting, and repairing real-world in-context-learning-based text-to-SQL errors. arXiv:2501.09310, 2025. Accepted at FSE 2026.

[9] Zijin Hong, Hao Chen, Zheng Yuan, Qinggang Zhang, Luyao Zhuang, Qing Liao, Feiran Huang, Yangqiu Song, and Xiao Huang. ErrorLLM: Modeling SQL errors for text-to-SQL refinement. In *KDD*, 2026. DOI: https://doi.org/10.1145/3770855.3818164.

[10] YiShan Zheng, Yuan Wu, and Yi Chang. ToolRobustBench: Stage-wise perturbation evaluation and failure diagnosis for tool-calling agents. arXiv:2608.23635, 2026.

[11] Dingzirui Wang, Longxu Dou, Xuanliang Zhang, Qingfu Zhu, and Wanxiang Che. DAC: Decomposed Automation Correction for text-to-SQL. In *Findings of EMNLP*, pages 385–402, 2025. DOI: https://doi.org/10.18653/v1/2025.findings-emnlp.22.

[12] Geonho Lee and Min-Soo Kim. SafeQL: Search-based refinement for safe and efficient LLM-based text-to-SQL. *Proc. VLDB Endow.*, 19(9):2210–2223, 2026. DOI: https://doi.org/10.14778/3819518.3819545.

[13] Jie Jiang, Siqi Shen, Haining Xie, Yang Li, Yu Shen, Danqing Huang, Bo Qian, Yinjun Wu, Wentao Zhang, Bin Cui, and Peng Chen. SQL-Governor: An LLM-powered SQL toolkit for real world application. arXiv:2509.08575, 2025.

[14] Jiaqian Wang, Yutao Qi, Wenjin Hou, Yu Pang, and Rui Yang. EGRE-FINE: An execution-grounded optimization framework for text-to-SQL schema refinement. arXiv:2605.00628, 2026.

[15] Xiang Huang, Sitao Cheng, Shanshan Huang, Jiayu Shen, Yong Xu, Chaoyun Zhang, and Yuzhong Qu. QueryAgent: A reliable and efficient reasoning framework with environmental feedback based self-correction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5014–5035, Bangkok, Thailand, 2024.

[16] Jipeng Cen, Jiaxin Liu, Zhixu Li, and Jingjing Wang. SQLFixAgent: Towards semantic-accurate text-to-SQL parsing via consistency-enhanced multi-agent collaboration. arXiv:2406.13408, 2024.

[17] Yicun Yang, Zhaoguo Wang, Yu Xia, Zhuoran Wei, Haoran Ding, Ruzica Piskac, Haibo Chen, and Jinyang Li. Automated validating and fixing of text-to-SQL translation with execution consistency. *Proc. ACM Manag. Data*, 3(3), Article 134, June 2025, 28 pages. DOI: https://doi.org/10.1145/3725271.

[18] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. Developing and benchmarking verification algorithms to improve text-to-SQL generation. *Proc. VLDB Endow.*, 19(11):3732–3744, 2026. DOI: https://doi.org/10.14778/3836663.3836721.

[19] Muhammad Adeel Ijaz. SQL Query Engine: A self-healing LLM pipeline for natural language to PostgreSQL translation. arXiv:2604.16511, 2026. Technical report.

[20] Yue Gong, Chuan Lei, Xiao Qin, Kapil Vaidya, Balakrishnan Narayanaswamy, and Tim Kraska. SQLens: An end-to-end framework for error detection and correction in text-to-SQL. arXiv:2506.04494, 2025.

[21] Anupreksha Jain and Manish Shrivastava. Reflect-SQL: A self-reflection based framework for text-to-SQL. In *PAKDD Workshops (Trends and Applications in Knowledge Discovery and Data Mining)*, LNCS 16603, pages 74–86. Springer, 2027. DOI: https://doi.org/10.1007/978-981-92-2014-4_7.

[22] Isabelle Mohr, Joao Pedro Gandarela, John Dujany, and Andre Freitas. Reflective reasoning for SQL generation. arXiv:2601.06678, 2026.

[23] Yoga Sri Varshan Varadharajan, Ajay Yadav, Ritesh Goru, Prateek Chaudhury, Constantine Caramanis, Prateek Jain, Divyateja Pasupuleti, and Sunil Kumar Pandey. A cost-aware agentic architecture for NL-to-SQL over nested enterprise schemas, with a new benchmark. arXiv:2609.04641, 2026.

[24] Yuan Tian and Tianyi Zhang. PV-SQL: Synergizing database probing and rule-based verification for text-to-SQL agents. In *Findings of ACL*, pages 25827–25845, 2026. DOI: https://doi.org/10.18653/v1/2026.findings-acl.1286.